Energy Intelligence Platform

End-to-end Machine Learning system for hourly electricity demand forecasting in France – from raw API data to a live interactive dashboard in production.

Machine Learning | MLOps | XGBoost | Streamlit | Data Engineering | Hugging Face Spaces

Why forecast *electricity demand*?

THE CORE CHALLENGE

Short-term forecasting (h+1) is critical for **grid operators**: it underpins real-time supply/demand balancing and renewable energy integration.

In France, RTE manages ~500 TWh/year across an interconnected European grid. A few percent forecasting error translates directly into imbalance costs and complicates the dispatch of intermittent renewables.

H+1 HORIZON

The most operationally relevant window for reserve activation and demand response. Target horizon of this project.

SOURCES : ENTSO-E + OPEN-METEO APIs

Actual national consumption data, public API, historical records from 2015.

583 MW

MAE - Validation (2024)

~1%

Of average national load

10 years

of training history

h+1

Forecast horizon

End-to-end system architecture

FULL PIPELINE

Ingestion → Preprocessing → Features → Training → Serving → Monitoring, orchestrated by **main.py**

01
INGESTION
Raw Parquet
Hive-partitioned

02
PREPROCESSING
Merge
Interpolation
Cleaning

03
FEATURES
Lags
Calendar
Aligned weather

04
MODELING
Ridge / XGB / LGBM
Temporal split

05
SERVING
Streamlit + Drift monitoring + HuggingFace Spaces + GitHub Actions

ENTSO-E TRANSPARENCY PLATFORM
Hourly electricity consumption for France (2015–2026)

OPEN-METEO API
Hourly temperatures + ~2h weather forecast (no API key required)

27 UNIT TESTS – ALL PASSING
pytest · ingestion · preprocessing · features · API · monitoring

Data pipeline

Five idempotent steps orchestrated via **--steps**.
Each step logs **[SKIP]** / **[FETCH]** / **[SAVED]** to guarantee reproducibility.

1

Historical Ingestion – ENTSO-E + Open-Meteo
Fetches 2015–2026 data via API. Idempotent guard clauses: if a year already exists on disk, it is automatically skipped.

2

Cleaning & Merging
Joins electricity and weather data on the time index. Interpolation strategy varies by column: linear for smooth variables, ffill for shortwave_radiation (legitimate nighttime zeros).

3

Feature engineering — 10 final variables
Lag features (t, t-1, t-24, t-168), calendar features (hour, day_of_week, week_of_year, is_weekday), weather (temperature_t, temperature_t-24). Real-time window: 192h (not 168h) to guarantee the t-168 lag row is always available.

4

Real-time snapshot + drift monitoring
Rolling 192h window from ENTSO-E. Open-Meteo weather forecast (~2h available, then NaN). Evidently drift report triggered automatically after each ingestion cycle.

Feature engineering

10 features structured across three families. XGBoost handles integer calendar features natively, no sin/cos cyclic encoding needed.

Lag features

load_t – current load
load_t-1 – 1 hour ago
load_t-24 – same hour yesterday
load_t-168 – weekly seasonality

Calendar features

hour – hour of day (0-23)
day_of_week – day (0=Monday)
week_of_year – ISO week
is_weekday – 0 if weekend or public holiday
is_holiday = is_weekday=0

Weather features

temperature_t – t+1 forecast (aligned -1h)
temperature_t-24 – temperature 24h ago

FEATURE_COLS as a single source of truth

Imported from **config.py** and shared between training and inference. Eliminates any risk of feature mismatch between training and serving.

Training & *results*

Strict temporal split, no future data leakage into training. Three models compared head-to-head.

Train	Validation	Test
2015-2023	2024	2024

Model	Val MAE (MW)	Val MAE (%)	Selected
Ridge Regression	~1090	–	–
Random Forest	~620	**~1.1%**	–
XGBoost	**583 MW**	**~1.0%**	**✓ best_model.pkl**

Feature importance (XGBoost)

load_t — 52.8% and **load_t-1 — 38.9%** → **91.7% of total importance** concentrated in 2 features.

Honest analysis – 2025 test set

Test MAE is significantly higher than validation MAE. This result is documented and explained, not hidden.

583 MW

MAE Validation (2024)

~5,606 MW

MAE Test (2025)

ABNORMALY COLD WINTER 2025

Consumption peaks outside the training distribution. The model underestimates demand at very low temperatures it had rarely seen during 2015–2023 training.

ATYPICAL SPRING AND AUTUMN

Unexpectedly low consumption across these periods, possibly linked to post-energy-crisis behavioral shifts. A distribution shift not captured by the 2015–2023 training data.

DELIBERATE PORTFOLIO CHOICE

A production system would integrate continuous retraining and automated shift detection. Documenting a limitation and explaining its root cause demonstrates more engineering maturity than concealing it behind a clean validation score.

MLOps – *drift monitoring*

At each real-time ingestion cycle, Evidently compares the current 24h snapshot against the 2024 reference distribution using a KS test.

MONITORED FEATURES (KS TEST, p < 0.05)

- load_t, load_t-1, load_t-24, load_t-168
- day_of_week
- is_weekday
- week_of_year
- hour — see false positive below ↓

WHY NOT WEATHER FEATURES?

Open-Meteo provides ~2h of weather forecast. Beyond that, temperature = NaN.

Monitoring a near-empty feature holds no statistical value.

⚠️ DOCUMENTED FALSE POSITIVE — hour

The 24-row snapshot produces a perfectly uniform [0–23] distribution, unlike the non-uniform 2024 reference. A structural artefact, not a real signal. Documented in project_assumptions_and_sources.md §11.7.

OUTPUT

Timestamped JSON reports saved to data/monitoring/, surfaced in the Model Performance tab of the Streamlit dashboard.

REST API & *serving layer*

FastAPI exposes two endpoints. Feature reconstruction is entirely server-side — no input required from the client to get a prediction.

GET
/health

```
status: "ok"
model_loaded: true
model_name: "xgboost"
feature_cols: [...]
```

GET
/predict

```
prediction_mw: 46 234.5
prediction_datetime_utc: "2025-03-29T10:00..."
model: "xgboost"
timestamp_utc: "2025-03-29T09:00..."
```

ARCHITECTURAL DECISIONS

- Features are reconstructed server-side from the real-time Parquet snapshot, no user input required to call /predict.
- **The API is not deployed on HF Spaces**: the dashboard reads Parquet files directly from disk. A deliberate, documented trade-off.
- The API remains in the repository as a portfolio artefact demonstrating REST serving knowledge.
- Interactive docs available at /docs via FastAPI's auto-generated Swagger UI.

Deployment – *Hugging Face Spaces*

Public dashboard accessible without an account, running in production via Docker SDK on HF Spaces.

1 **Local pipeline**
python main.py --steps realtime generates a fresh data snapshot.

2 **HF upload**
python upload_to_hf.py pushes code, models and data via HfApi.upload_folder.

3 **Automatic Docker build**
HF rebuilds the container (~1–2 min). Port 7860, python:3.12-slim base image.

4 **runpy entry point**
app.py injects src/ and dashboard/ into sys.path before launching Streamlit.

5 **GitHub Actions**
CI workflows for automated synchronisation from GitHub → HF Spaces.

LIVE DASHBOARD

huggingface.co/spaces/bachirij/energy-intelligence-platform

Running, public access, no account needed.

Tech stack & key takeaways

ML & DATA

- Python 3.12
- XGBoost
- Scikit-Learn
- Evidently
- Pandas
- Parquet

SERVING & API

- FastAPI
- Uvicorn
- Pydantic
- APSCheduler
- Streamlit
- Plotly

INFRA & DEPLOY

- Docker
- GitHub Actions
- HF Spaces
- huggingface_hub
- supervisord

DATA SOURCES & TOOLS

- ENTSO-E API
- Open-Meteo
- holidays
- pyarrow
- pytest
- conda

▸ LIVE DASHBOARD

▸ GITHUB REPO

PRODUCTION-GRADE ML PIPELINE

Idempotence, separation of concerns, single source of truth for features across training and serving.

TRANSPARENCY OVER POLISH

The 2025 distribution shift is analysed and explained, not buried behind a clean validation number.

REAL MLOPS

Automated drift monitoring, timestamped reports, documented false positives, not just a checkbox.

FULL END-TO-END STACK

From raw API data to a public production dashboard, including REST serving and containerised deployment.